Accès aux données Serveur

Les applications web modernes sont souvent amenées à récupérer et afficher dynamiquement des données (texte, images, vidéos, …) provenant de différentes sources possibles : une base de données, un serveur de fichiers, une API tierce, …

Ce chapitre aborde les fondamentaux de l'accès à des données serveur à partir d'une application javascript. Pour illustrer ces concepts, nous nous concentrons sur l'accès à une API REST tierce à l'aide de l'API fetch de Javascript et le format JSON pour l'échange de données.

I. Principe de la communication avec le serveur

Dans une application web dynamique, le navigateur (partie client) communique, généralement, avec le serveur via le protocole HTTP : HyperText Transfer Protocol comme le synthétise le schéma suivant :

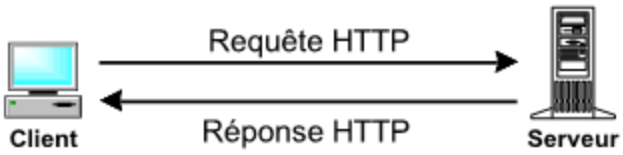


Figure 1: Illustration de la communication avec le serveur

Ainsi :

- Le client envoie une requête HTTP (HTTP Request) vers le serveur qui héberge l'application (lors de la soumission d'un formulaire, du lancement d'une recherche, …)
- Le serveur traite la requête et renvoie une réponse (HTTP Response).

I.1. La requête HTTP

Une requête HTTP contient des informations envoyées par le client au serveur pour demander une ressource spécifique.

I.1.1. Structure d'une requête HTTP

Une requête HTTP comprend plusieurs parties, notamment :

- **La ligne de requête** : qui contient la méthode HTTP utilisée (GET, POST, PUT, etc.), l'URL de la ressource demandée, ainsi que la version du protocole HTTP utilisée.
- **Les en-têtes de requête** : qui contiennent des informations supplémentaires sur la requête, telles que le type de contenu accepté par le client, les informations d'authentification, etc.

- **Le corps de la requête** : qui peut contenir des données supplémentaires envoyées par le client tel que les données à ajouter pour une requête de type POST.

Exemple :

En faisant une requête GET vers l'URL https://api.github.com/users, l'inspection de la requête dans le navigateur donne :



Figure 2: Extrait des informations sur la requête HTTP

I.1.2. Principales méthodes d'une requête HTTP

Les méthodes HTTP désignent les opérations que le client désire réaliser. Le tableau suivant synthétise les principales méthodes http.

Tableau 1: Principales méthodes HTTP

Méthode	Rôle
GET	Récupère des données à partir d'une ressource spécifiée
POST	Transmet des données à traiter pour une ressource spécifiée. Elle peut créer une ressource ou modifier une ressource existante
PUT	Remplace ou met à jour une ressource spécifiée avec des données
DELETE	Supprime une ressource spécifiée

I.2. La réponse HTTP

La réponse HTTP est envoyée par le serveur vers le client suite à une requête http.

I.2.1. Structure d'une réponse HTTP

Une réponse http comporte les 3 parties suivantes :

- **Le statut de la réponse** : qui comporte un code de statut et un message associé.
- **L'en-tête de la réponse** : qui contient des informations sur la réponse telles que la date de la réponse, le type de contenu, les informations de sécurité, …

- **Le corps de la réponse :** qui contient les données envoyées par le serveur, il peut inclure :
  - ❖ Du code HTML de la page qui sera interprétée puis affichée par le navigateur
  - ❖ Des données (au format JSON, XML, …)
  - ❖ Du texte, du CSV, …

Exemple :

La réponse à une requête GET vers l'URL https://api.github.com/users, donne :

Tableau 2: Aperçu d'un extrait de la réponse HTTP

Entête de la réponse	Corps de la réponse
▾ Général URL de requête: https://api.github.com/users Mode de requête: GET Code d'état: 304 Adresse distante: 140.82.121.5:443 Règlement sur les URL de provenance: strict-origin-when-cross-origin ▾ En-têtes de réponse access-control-allow-origin: * access-control-expose-headers: ETag, Link, Location, Retry-After, X- X-RateLimit-Used, X-RateLimit-Resource, X-RateLimit-Reset, X-OAut -GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecatic cache-control: public, max-age=60, s-maxage=60 content-encoding: gzip content-security-policy: default-src 'none' content-type: application/json; charset=utf-8	[{ "login": "mojombo", "id": 1, "node_id": "MDQ6VXNlcjE=", "avatar_url": "https://avatars.githubusercontent "gravatar_id": "", "url": "https://api.github.com/users/mojombo", "html_url": "https://github.com/mojombo", "followers_url": "https://api.github.com/users/r "following_url": "https://api.github.com/users/r "gists_url": "https://api.github.com/users/mojor "starred_url": "https://api.github.com/users/mo "subscriptions_url": "https://api.github.com/use "organizations_url": "https://api.github.com/use "repos_url": "https://api.github.com/users/mojor "events_url": "https://api.github.com/users/moj "received_events_url": "https://api.github.com/r "type": "User", "site_admin": false },

I.2.2. Les statuts des réponses HTTP

HTTP définit 40 codes de statut répertoriés comme le synthétise le tableau suivant :

Code	Signification
1XX	Message d'information *Exemple :* **101** Demande de changement de protocole
2XX	Succès de la requête client *Exemple :* **201** Document créé
3XX	Redirection de la requête client *Exemple :* **302** Le document a changé d'adresse temporairement
4XX	Requête client incomplète *Exemple :* **404** document introuvable
5XX	Erreur du serveur *Exemple :* **500** Erreur inattendue au niveau du serveur

II. Accès à des données serveur avec Javascript

Lors du développement d'une application web qui récupère des données à partir d'une source distante, il est nécessaire de déterminer :

- La source de données
- Le format d'échange de données
- La technique d'accès aux données avec javascript

Pour la suite du cours, le choix de chacun de ces 3 éléments est présenté dans cette section.

II.1. Source de données Serveur

Une application web dynamique peut accéder à des données distantes provenant de sources différentes telles que :

- *Bases de données distantes* : Bases de données qui sont hébergées sur un serveur distant.
- *Services cloud* : qui fournissent des services de stockage de données et d'accès à distance tels que Amazon Web Services (AWS), Google Cloud Platform ou Microsoft Azure.
- *Serveurs de fichiers* : fichiers stockés sur des serveurs distants.
- *API tierces* : interfaces de programmation permettant d'accéder à des données de services tiers
- …

Les principes fondamentaux d'accès à une source de données distante, tels que les requêtes HTTP et le traitement de la réponse, sont globalement similaires pour les différentes sources de données.

Dans ce cours, nous nous concentrerons sur l'accès à des données récupérées à partir d'une **API tierce.** Nous avons choisi cette approche pour nous limiter à une implémentation côté client et pour illustrer de manière concrète comment accéder aux données d'une source distante en utilisant JavaScript.

II.1.1. Qu'est-ce qu'une API ?

Une API (Application Programming Interface) est une interface de programmation permettant à des applications de communiquer entre elles et de s'échanger mutuellement des services ou des données.

Elle joue ainsi le rôle d'intermédiaire entre deux applications qui ont besoin de partager des informations, c'est d'ailleurs généralement le cas de la communication entre la partie frontend et backend qui ne se fait pas directement, mais, via une API.

Exemples :

- L'API de *Google Maps* fournit des fonctionnalités de cartographie, de géolocalisation, de recherche des lieux, …
- L'API de *Facebook* fournit une interface pour accéder aux données de Facebook, telles que les profils d'utilisateurs, les pages, les publications et les commentaires
- L'API de *OpenWeatherMap* fournit des informations météorologiques en temps réel et des prévisions pour les villes du monde entier.

II.1.2. Principaux types d'API

Il existe différents types d'API dont il est possible de mentionner :

- Les ***API REST*** : REST (Representational State Transfer). Ces API utilisent les méthodes HTTP (GET, POST, PUT, DELETE) pour accéder à des ressources (par exemple, une liste de produits, un utilisateur) et des codes de réponse HTTP pour indiquer le succès ou l'échec de la requête.
- Les ***API SOAP*** : SOAP (Simple Object Access Protocol) est un protocole de communication basé sur XML. Les API SOAP sont souvent utilisées dans les environnements d'entreprise pour échanger des données entre des applications.
- Les ***API GraphQL*** : qui permettent à travers le langage de requête GraphQL de spécifier exactement les données à extraire en définissant explicitement les champs et les relations à récupérer.

Pour la suite du cours, nous adopterons les API REST qui sont les plus utilisées actuellement.

II.1.3. Caractéristiques d'une API REST

Une API est dite REST si elle respecte le style Architectural REST en respectant les 6 propriétés suivantes :

- **Architecture client-serveur** : constituée de clients, de serveurs et de ressources, avec des requêtes gérées via http. Les clients et les serveurs sont indépendants et ne connaissent que les URL des ressources.
- **Interface Uniforme** : qui permet un transfert standardisé des informations
- **Absence d'état** : chaque requête doit inclure toutes les informations nécessaires à son traitement.
- **Capacité de mise en cache**. Lorsque cela est possible, les ressources doivent pouvoir être mises en cache côté client ou serveur pour rationaliser les interactions client/serveur.
- **Système en couches** : les appels et les réponses passent par différentes couches.
- **Code à la demande** (facultatif) : les réponses peuvent éventuellement contenir du code exécutable (tels que des applets Java), dans ces cas, le code ne doit s'exécuter qu'à la demande

II.1.4 Les API REST et les URI

Toute donnée qu'une API web peut fournir est une ressource.

Exemple :

> Pour l'API de Facebook, un utilisateur, une page, une image ou un commentaire représentent des ressources

Une API REST se base sur les URI (Uniform Resource Identifier) pour identifier une ressource. Les URI sont construites de façon hiérarchique avec une certaine sémantique.

Exemples :

On considère l'API de test publique et gratuite suivante :

`https://jsonplaceholder.typicode.com/`

- Liste des utilisateurs

`https://jsonplaceholder.typicode.com/users`

- Utilisateur dont le username est « Antonette »

`https://jsonplaceholder.typicode.com/users?username=Antonette`

- Liste des publications

`https://jsonplaceholder.typicode.com/posts`

- Publication d'id 3

`https://jsonplaceholder.typicode.com/posts/3`

- Liste des commentaires de la publication d'id 1

`https://jsonplaceholder.typicode.com/posts/1/comments`

Remarque :
Les 5 URL précédentes qui permettent d'accéder à une ressource sont nommés des **endpoints**

A noter que l'action définie sur une ressource dépend de la méthode de requête (GET, POST, PUT, DELETE)

Exemples :

- Récupérer la liste des publications :

`GET https://jsonplaceholder.typicode.com/posts`

- Créer une publication (en spécifiant la publication dans le corps de la requête)

`POST https://jsonplaceholder.typicode.com/posts`

- Supprimer la publication d'id 1

`DELETE https://jsonplaceholder.typicode.com/posts/1`

II.2. Format des données échangées

Les données récupérées à partir d'une API REST ont, le plus souvent, l'un des 2 formats suivants :

- JSON : JavaScript Object Notation
- XML : eXtensible Markup Language

Pour la suite du cours, nous adopterons le format JSON qui est le plus couramment utilisé en raison de sa simplicité, sa lisibilité et sa compatibilité avec la plupart des langages de programmation courants.

II.2.1. Présentation du format JSON

Le format JSON est un format standard utilisé pour représenter des données structurées de façon semblable aux objets Javascript. Les données au format JSON sont sauvegardées dans un fichier ayant pour extension **.json**.
Il est utilisé pour :

- La sauvegarde des données (couramment pour des fichiers de configuration)
- L'échange de données à travers le réseau entre le client et le serveur
- Modéliser des données sous forme textuelle

II.2.2. Structure et syntaxe d'un fichier JSON

Un fichier JSON est délimité par :

- Une accolade ouvrante **{** et une accolade fermante **}** si c'est 1 seul objet JSON
- Un crochet ouvrant **[** et un crochet fermant **]** si c'est un tableau d'objets JSON

Un objet JSON est un ensemble non ordonné de paires clé-valeur qui respectent la notation suivante :

- La clé est une chaîne de caractères délimitée par des guillemets doubles « " »
- Les valeurs acceptées sont les mêmes types de données de base d'un objet Javascript : chaînes de caractères, nombres, tableaux, booléens, objet.
- Les paires clé-valeur sont séparées par des virgules « , »

Exemple :

```
{
    "etablissement":"ISET Charguia",
    "etatique":true,
    "nbEtudiants":980,
    "departements":["TI", "SEG"],
    "specialites":{
        "nombre":7,
        "details":[
            {"departement":"TI", "parcours":["DSI","RSI","SEM"] },
            {"departement":"SEG", "parcours":["MGT","MAF","LT","MIN"] }
        ]
    }
}
```

II.2.3. Méthodes pour manipuler un objet JSON

Les principales méthodes pour manipuler un objet JSON sont :

- **JSON.parse(texte)**

Méthode qui analyse une chaîne de caractères JSON et construit la valeur JavaScript ou l'objet décrit par cette chaîne

Exemple :

```
let ch = '{"nom":"Ali", "age":15}';
let objetJSON = JSON.parse(ch);
console.log(objetJSON);      // {nom: 'Ali', age: 15}
```

- **JSON.stringify(valeurJS)**

Méthode qui convertit une valeur javascript en chaîne JSON.

Exemple :

```
let chJson= JSON.stringify({nom:'Eya', age:20, etudiant:true});
console.log(chJson);     //  {"nom":"Eya","age":20,"etudiant":true}
console.log(JSON.stringify(new Date(2023,3,24)));    // "2023-04-23T22:00:00.000Z"
console.log(JSON.stringify(['note', undefined, 15]));  //  ["note",null,15]
```

II.3. Accès aux données serveur avec Javascript : techniques et programmation asynchrone

Il existe plusieurs techniques pour accéder aux données serveur avec JavaScript qui sont toutes basées sur la programmation asynchrone vu la nature de l'échange des données avec un serveur distant qui peut prendre du temps.

II.3.1. Principales techniques d'accès aux données

Javascript offre différentes méthodes d'accès à des données serveur dont il est possible de citer principalement :

- **AJAX** : Asynchronous Javascript and XML qui récupère des données asynchrones en utilisant l'objet XMLHttpRequest. Cette méthode a largement été utilisée au passé afin de récupérer des données en arrière-plan sans recharger la page.
- L'**API Fetch** : fournit une interface JavaScript pour récupérer et envoyer des données à un serveur via des requêtes HTTP.
- Les **Websockets** : technologie de communication en temps réel bidirectionnelle pour la communication entre le client et le serveur
- La bibliothèque **Axios** : une bibliothèque JavaScript populaire qui fournit des fonctionnalités pour effectuer des requêtes HTTP à partir d'un navigateur ou d'un serveur Node.js.

Pour la suite du cours, nous nous focaliserons sur l'api **Fetch** vu qu'elle offre les avantages suivants :

- L'API Fetch est **native** : elle est intégrée dans les navigateurs modernes et ne nécessite pas d'installation d'une bibliothèque tierce comme pour AJAX ou Axios
- L'API Fetch est très **légère** et **concise** : il ne faut pas beaucoup de code pour exécuter une requête
- L'API Fetch utilise les **promesses** pour gérer les réponses de la requête : la gestion des réponses asynchrones est facilitée et plus lisible
- L'API Fetch est plus **récente** et plus adaptée aux applications web modernes.

II.3.2. Programmation asynchrone pour l'accès au serveur

Les requêtes http sont souvent des opérations qui prennent du temps, ce qui peut impacter négativement sur l'expérience utilisateur.

C'est pour cela que les différentes techniques d'accès aux données serveur sont basées sur la **programmation asynchrone** qui est **non bloquante** c'est-à-dire que le programme continue à s'exécuter pendant que les requêtes sont en cours d'exécution.

- **Programmation synchrone vs Programmation asynchrone**

Le tableau comparatif suivant liste les principales différences entre la programmation synchrone et asynchrone.

Tableau 3: Comparaison programmation synchrone/asynchrone

Programmation synchrone	Programmation asynchrone
Les tâches sont exécutées séquentiellement : l'une après l'autre.	Les tâches peuvent s'exécuter en parallèle sans bloquer l'exécution du programme
Le programme attend que chaque instruction soit terminée pour passer à la suivante	Le programme continue à s'exécuter pendant que des tâches sont en cours d'exécution : il ne bloque pas si une méthode asynchrone est évoquée.
Les erreurs peuvent interrompre l'exécution	Les erreurs peuvent être gérées sans interrompre l'exécution

Exemple :

La méthode ***setTimeout*** est une méthode **asynchrone** (non bloquante). Le résultat de l'exécution de ces 3 instruction donne :

Code

```
console.log('Début');
setTimeout(()=>console.log('Après 1 seconde'), 1000);
console.log('Fin');
```

Résultat

```
Début
Fin
Après 1 seconde
```

- **Utilisation des promesses pour la programmation asynchrone**

L'API fetch fonctionne en asynchrone en utilisant les **promesses** (promise en anglais). Une promesse est un objet qui représente la complétion ou l'échec d'une opération asynchrone.

En d'autres termes, il renvoie une valeur qui peut être disponible immédiatement, dans le futur ou jamais.

Exemple :

L'exemple suivant illustre la différence entre la valeur renvoyée par une fonction synchrone et asynchrone (déclarée avec le mot clé **`async`**)

Fonction synchrone

```
function f(){
    return 5;
}
```

Fonction asynchrone

```
async function g(){
    return 5
}
```

```
console.log(f());
```

Résultat:

```
5
```

```
console.log(g());
```

Résultat:

```
▼Promise {<fulfilled>: 5}
  ▶[[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: 5
```

Pour exécuter un traitement une fois la promesse résolue avec succès, il faut appeler la méthode **then**

Exemple :

Dans l'exemple précédent, pour afficher la valeur renvoyée par la fonction g, il faut écrire :

```
g()
.then( res => console.log(res))    // Affiche 5
```

Remarque :
Il est possible de chaîner plusieurs promesses si la méthode **then()** renvoie à son tour une promesse.

III. Utilisation de l'API Fetch

L'API Fetch fournit la méthode *fetch*, ainsi que les interfaces *Headers*, *Request* et *Response* pour effectuer des requêtes http et récupérer des ressources à partir d'un serveur.

III.1. Syntaxe de base pour l'utilisation de la méthode fetch

L'API Fetch de Javascript fournit une méthode clé **fetch()** qui permet l'échange de données avec le serveur de manière asynchrone. La méthode *fetch*:

- prend un argument obligatoire : l'url vers la ressource
- renvoie une **promesse** JavaScript qui résout la réponse à cette demande, qu'elle aboutisse ou non.

Syntaxe :

```
fetch(url [, options])
      .then (response =>{
            // Traitement de la réponse qui renvoie une promesse
            // Exemple : response.json()
                        })
      .then(data => {
            // Utilisation des données JSON
                        })
      .catch(error =>{
            // Gestion des erreurs : Ce bloc n'est pas obligatoire
                        })
```

Remarques :

- Les **`options`** sont un objet facultatif qui contient : les paramètres de la requête tels que : la méthode http utilisée (GET, POST, …), l'entête et le corps de la requête
- **`then()`** permet d'exécuter une action lorsque la promesse est résolue avec succès
- L'instruction **`response.json()`** extrait les données JSON à partir de la réponse http. Elle renvoie à son tour une promesse.
- **`catch()`** permet de capturer les erreurs qui se produisent et les gère. L'appel de cette méthode n'est pas impératif

Exemple 1:
Soit l'api de test JSONPlaceholder qui offre l'endpoint suivant :

`https://jsonplaceholder.typicode.com/users`

On souhaite récupérer la liste de tous les utilisateurs et afficher dans la fenêtre console :

- la liste des utilisateurs
- le nombre d'utilisateurs
- Les informations du premier utilisateur

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => {
       console.log(data);
       console.log(data.length);
       console.log(`Nom:${data[0].name} - Email: ${data[0].email}`)
     })
    .catch(error=>console.log(error))
```

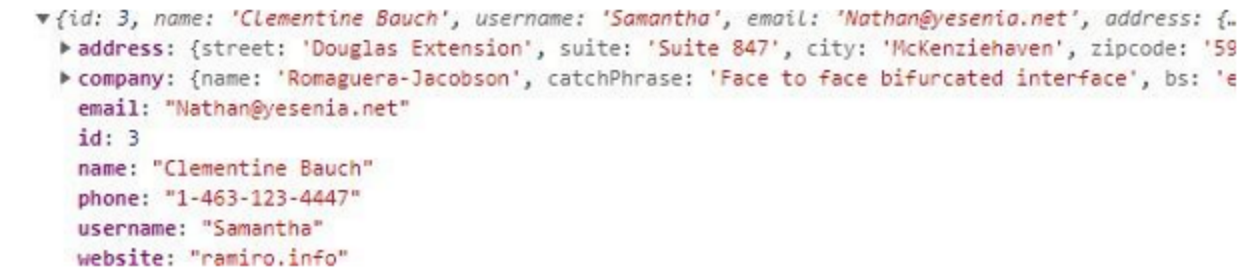
L'affichage donne :

```
▼(10) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
  ▶0: {id: 1, name: 'Leanne Graham', username: 'Bret', email: 'Sincere@april.biz', address: {…},
  ▶1: {id: 2, name: 'Ervin Howell', username: 'Antonette', email: 'Shanna@melissa.tv', address:
  ▶2: {id: 3, name: 'Clementine Bauch', username: 'Samantha', email: 'Nathan@yesenia.net', addre
  ▶3: {id: 4, name: 'Patricia Lebsack', username: 'Karianne', email: 'Julianne.OConner@kory.org',
  ▶4: {id: 5, name: 'Chelsey Dietrich', username: 'Kamren', email: 'Lucio_Hettinger@annie.ca', a
  ▶5: {id: 6, name: 'Mrs. Dennis Schulist', username: 'Leopoldo_Corkery', email: 'Karley_Dach@ja
  ▶6: {id: 7, name: 'Kurtis Weissnat', username: 'Elwyn.Skiles', email: 'Telly.Hoeger@billy.biz',
  ▶7: {id: 8, name: 'Nicholas Runolfsdottir V', username: 'Maxime_Nienow', email: 'Sherwood@rosa
  ▶8: {id: 9, name: 'Glenna Reichert', username: 'Delphine', email: 'Chaim_McDermott@dana.io', a
  ▶9: {id: 10, name: 'Clementina DuBuque', username: 'Moriah.Stanton', email: 'Rey.Padberg@karin
   length: 10
  ▶[[Prototype]]: Array(0)
10
Nom:Leanne Graham - Email: Sincere@april.biz
```

Exemple 2 :
On souhaite tester le cas d'une url dynamique.
Pour cela, on souhaite récupérer l'utilisateur dont l'id est fourni dans l'URL et afficher ses informations dans la fenêtre console.

```
let id = 3;
const url = 'https://jsonplaceholder.typicode.com/users';
fetch(`${url}/${id}`)
     .then(response => response.json())
     .then(data => console.log(data))
     .catch(error=>console.log(error))
```

L'affichage donne :



III.2. Utilisation de la méthode fetch avec des paramètres de requête

La recherche d'une ressource bien spécifique (utilisateur ayant un email précis, la monnaie d'un pays donné, ...) peut nécessiter des paramètres au niveau de la requête.

Les paramètres de requête dans une requête fetch font référence aux informations supplémentaires pouvant être incluses dans l'URL de la requête pour spécifier certaines conditions ou filtrer les résultats.

Syntaxe de l'URL:

Les paramètres de requête sont généralement ajoutés à la fin de l'URL, après un point d'interrogation « ? », et sont séparés par des ampersands « & ». Chaque paramètre de requête se compose d'une clé et d'une valeur, séparées par un signe égal « = ».

```
URL?clé1=valeur1&clé2=valeur2
```

Exemple :

On souhaite récupérer l'utilisateur dont l'email est passé en paramètre dans l'URL et afficher ses informations dans la fenêtre console.

```
 let email = "Nathan@yesenia.net";
 const url = 'https://jsonplaceholder.typicode.com/users';
//fetch('https://jsonplaceholder.typicode.com/users?email=Nathan@yesenia.net')
 fetch(`${url}?email=${email}`)
     .then(response => response.json())
     .then(data => console.log(data)
     .catch(error => console.log(error))
```

L'affichage donne :

```
Clementine Bauch
```

III.3. Utilisation des options dans la méthode fetch

Il est possible de définir dans la méthode *fetch*, un second paramètre optionnel qui permet de définir un certain nombre de réglages.

Il existe plusieurs options possibles dont les plus couramment utilisées sont synthétisées dans le tableau suivant :

Tableau 4: Quelques options de la méthode fetch

Option	Description
method	Méthode utilisée pour la requête *Exemples* : GET (valeur par défaut), POST, PUT, DELETE, …

headers	Entêtes à ajouter à la requête qui comportent des informations sur l'authentification, le type de contenu envoyé ou reçu….
	Exemple :
	`headers: {`
	`'Content-Type': 'application/json'`
	`}`

body	Corps de la requête *Exemple :* les données d'un utilisateur à ajouter lors d'une requête POST
mode	Mode de fonctionnement de la requête. *Exemples* : cors, no-cors, same-origin, …

Exemple:

On souhaite récupérer la liste des aéroports où opère la compagnie aérienne Ryanair, ainsi que leur ville, région, pays, latitude, longitude, nom et codes IATA à travers l'endpoint suivant :

`https://ryanair.p.rapidapi.com/airport`

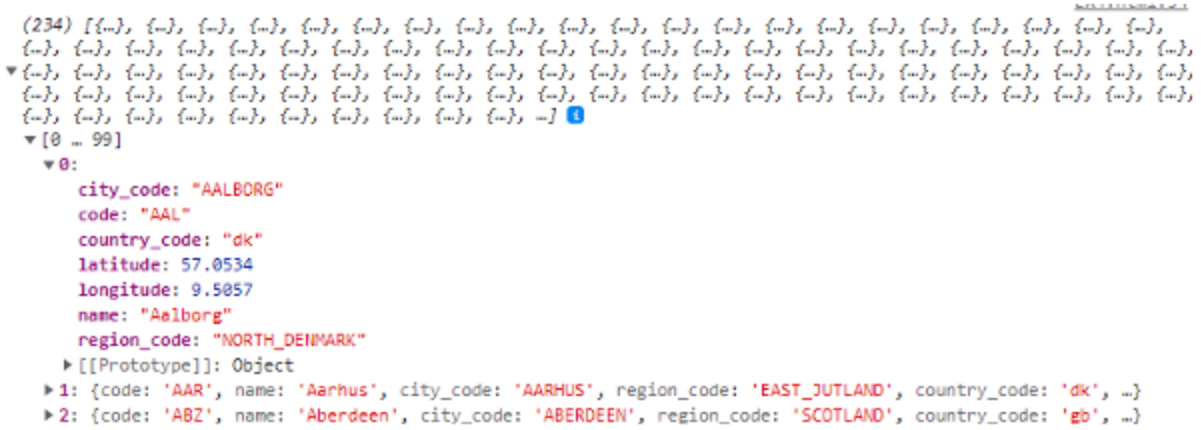
Cette API nécessite une authentification dont la clé est fournie suite à une souscription.

La clé (ici : `255c99d668mshb475ec51acca46ap1353d0jsnf59640bae26c`) est fournie dans l'entête de la requête http.

```
const url = 'https://ryanair.p.rapidapi.com/airports';
const options = {
     method: 'GET',
     headers: {
     'X-RapidAPI-Key': '255c99d668mshb475ec51acca46ap1353d0jsnf59640bae26c',
     'X-RapidAPI-Host': 'ryanair.p.rapidapi.com'
            }
        };                                          // Clé d'authentification

fetch(url, options)
     .then(response => response.json())
     .then(data => console.log(data))
     .catch(error=>console.log(error))
```

L'exécution renvoie les informations des 234 aéroports dont l'extrait suivant :


(234) [{…}, {…}, {…}, {

```
async function displayUsersAsync(){
 try{
      let response = await fetch("https://jsonplaceholder.typicode.com/users")
      let data = await response.json();
      console.log(data);
 }
 catch(error){
     console.log(error)
 }
}
```

Remarque :
Il est à noter que l'utilisation de *await* **ne bloque pas** le fil d'exécution. Au lieu de cela, le programme passe à l'instruction suivante pendant que la requête est en cours.

Lorsque la réponse est prête, le programme revient à l'instruction *await* et continue l'exécution.

L'exécution de ces 3 instructions donne :

```
console.log("Début");
displayUsersAsync();
console.log("Fin");
```

```
Début
Fin
▸Fetch a fini de se charger : GET "https://jsonplaceholder.typicode.com/users".
▸(10) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
```